

Cidades ESG Inteligentes

Coleta+ ESG API

Pipeline CI/CD, containerização e deploy automatizado em ambientes de staging e produção

Integrante	[Seu nome — RM]
Stack	Java 21 · Spring Boot 4 · Oracle 23 Free
DevOps	GitHub Actions · Docker · Docker Compose · GHCR · Azure
Repositório	github.com/justinojosesilva/coleta-plus

1. Descrição do pipeline

O Coleta+ é uma API REST de gestão de resíduos em pontos de coleta urbanos: registra resíduos e coletas, gera alertas automáticos quando um ponto atinge 90% da capacidade e calcula indicadores ESG (volume reciclado, pontos sobrecarregados e eficiência de coleta). Nesta fase o projeto passou a ter um ciclo de vida totalmente automatizado, do commit até a produção.

Ferramentas

Ferramenta	Papel
GitHub Actions	Orquestra o pipeline (<code>.github/workflows/ci-cd.yml</code> + workflow reutilizável <code>deploy.yml</code>).
Maven + JUnit 5 + Mockito	Build e testes: Surefire para os unitários e Failsafe para os de integração.
Oracle 23 Free (service container)	Banco real usado pelo teste de integração dentro do runner.
Docker Buildx + GHCR	Gera a imagem e publica no GitHub Container Registry.
GitHub Environments	<code>staging</code> e <code>production</code> , com secrets próprios; produção exige aprovação manual e só aceita a branch <code>main</code> .
Azure Virtual Machine	Ubuntu 24.04 (Standard_B2ms) com Docker; hospeda os dois ambientes isolados.

Etapas



Lógica de funcionamento

- **Gatilhos:** todo *push* na `main` executa o fluxo completo; *pull requests* executam só build e testes, validando a mudança antes do merge. Também pode ser disparado manualmente.
- **Build & testes:** compila com Java 21 e executa `./mvnw verify`. Os testes unitários cobrem regras de negócio (baixa de estoque na coleta, alerta de capacidade de 90%, cálculo de eficiência ESG), geração/validação de JWT e mapeamento de papéis. O teste de integração sobe a aplicação inteira contra um Oracle real, roda as migrations do Flyway e valida com `ddl-auto=validate` que todas as entidades batem com o schema. Relatórios e o `.jar` ficam como artefatos.
- **Imagem:** construída uma única vez e marcada com o hash do commit. *Build once, deploy many:* staging e produção recebem exatamente o mesmo artefato.

- **Deploy:** o runner gera o arquivo `<ambiente>.env` a partir dos secrets do environment, copia os arquivos por SSH para a VM e executa `deploy/deploy.sh`, que faz `docker compose pull + up -d` e aguarda o container ficar *healthy*.
- **Smoke test:** após cada deploy, o pipeline chama `/actuator/health`, confere em `/actuator/info` se o ambiente e a versão são os esperados, faz login e consulta os indicadores ESG. Qualquer falha interrompe o fluxo e produção não é liberada.
- **Segurança:** senhas do banco, segredo JWT e chave SSH ficam em GitHub Secrets. Nada sensível é versionado, e o banco de staging/produção não expõe porta para fora da VM.

2. Docker

Arquitetura

GitHub Actions → publica `ghcr.io/justinojosesilva/coleta-plus:sha-xxxxxxx` → acessa a VM via SSH

VM Azure · Ubuntu 24.04 · Docker Engine (portas públicas: 22, 8080, 8081)

Projeto Compose `coleta-staging`

app (Spring Boot) · porta **8081**→8080

oracle-db (Oracle 23 Free) · sem porta pública

rede backend · volume oracle-data

Projeto Compose `coleta-production`

app (Spring Boot) · porta **8080**→8080

oracle-db (Oracle 23 Free) · sem porta pública

rede backend · volume oracle-data

Cada ambiente é um projeto Docker Compose independente, com rede, volume e banco próprios. Um deploy em staging nunca afeta os dados de produção. A API encontra o banco pelo nome do serviço (`oracle-db`) na rede interna e só sobe quando o healthcheck do banco passa (`depends_on: service_healthy`).

Dockerfile (multi-stage)

```
# Estágio 1: build (JDK + Maven), não vai para produção
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /workspace
COPY pom.xml mvnw ./
COPY .mvn .mvn
RUN ./mvnw -B -q dependency:go-offline          # camada de dependências em cache
COPY src src
RUN ./mvnw -B -q -DskipTests package && cp target/coleta-plus-*.jar app.jar

# Estágio 2: runtime (somente JRE)
FROM eclipse-temurin:21-jre
RUN groupadd --system app && useradd --system --gid app --home /app app
WORKDIR /app
COPY --from=build --chown=app:app /workspace/app.jar app.jar
USER app                                # processo sem root
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError"
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=5s --start-period=90s --retries=5 \
  CMD curl -fs http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]
```

Imagem criada

Item	Valor
Registry / nome	ghcr.io/justinojosesilva/coleta-plus
Tags	sha-<commit> (imutável, usada nos deploys) e latest
Base de runtime	eclipse-temurin:21-jre (Ubuntu)
Tamanho	~425 MB (só JRE + .jar; o estágio de build com JDK/Maven fica de fora)
Usuário	app (não-root)
Healthcheck	/actuator/health (inclui conexão com o banco)

Comandos usados

Comando	Finalidade
cp .env.example .env	Criar a configuração local
docker compose up -d --build	Compilar a imagem e subir app + banco localmente
docker compose ps / logs -f app	Ver status (healthy) e logs
docker compose down -v	Derrubar o ambiente e apagar o volume
docker build -t coleta-plus:local .	Build manual da imagem
docker compose -p coleta-staging -f docker-compose.deploy.yml --env-file staging.env up -d	Deploy de um ambiente na VM (executado pelo deploy.sh)
./infra/create-azure-vm.sh	Criar a VM no Azure com Docker (cloud-init)

3. Pipeline em execução

PRINT PENDENTE: Visão geral do pipeline: build → imagem → staging → produção (prints/01-pipeline.png)

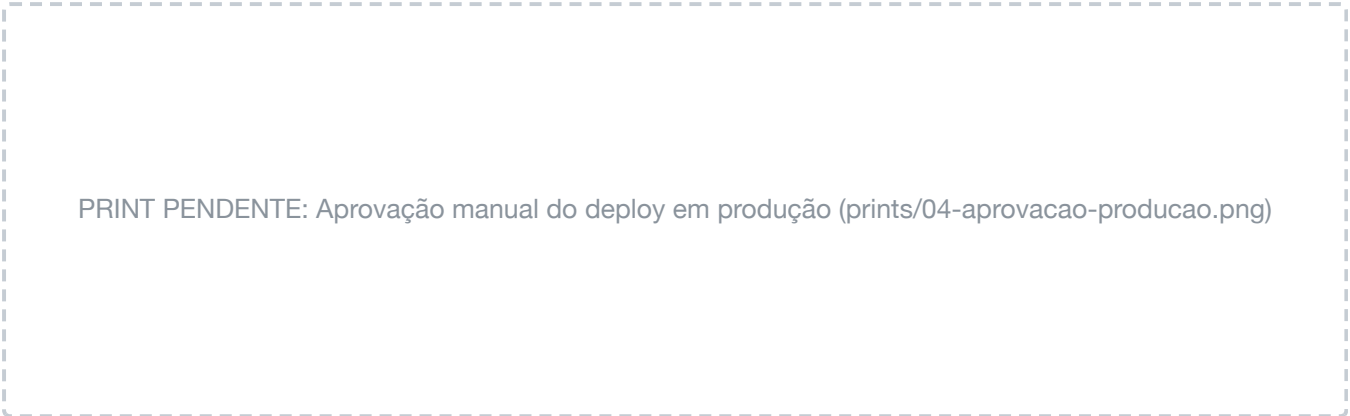
Figura 1: execução completa do workflow CI/CD no GitHub Actions.

PRINT PENDENTE: Job de build e testes (resumo dos testes) (prints/02-build-testes.png)

Figura 2: build e testes automatizados (unitários + integração com Oracle).

PRINT PENDENTE: Imagem publicada no GHCR (prints/03-imagem-ghcr.png)

Figura 3: imagem Docker publicada no GitHub Container Registry.



PRINT PENDENTE: Aprovação manual do deploy em produção (prints/04-aprovacao-producao.png)

Figura 4: gate de aprovação manual antes do deploy em produção.

4. Ambientes de staging e produção

PRINT PENDENTE: Staging: Swagger / health em http://IP:8081 (prints/05-staging.png)

Figura 5: ambiente de **staging** (porta 8081) em funcionamento.

PRINT PENDENTE: Produção: Swagger / health em http://IP:8080 (prints/06-producao.png)

Figura 6: ambiente de **produção** (porta 8080) em funcionamento.

PRINT PENDENTE: docker ps na VM mostrando os 4 containers (prints/07-containers-vm.png)

Figura 7: containers dos dois ambientes rodando na VM do Azure.

PRINT PENDENTE: Ambiente local com docker compose (prints/08-local-compose.png)

Figura 8: ambiente local com Docker Compose (app + Oracle, ambos healthy).

5. Desafios encontrados e soluções

1. Todos os endpoints protegidos retornavam 403, inclusive para o administrador

O smoke test do deploy (login + consulta de indicadores) revelou um bug que existia desde a fase anterior: o `CustomUserDetailsService` registrava a permissão como `ADMIN`, mas

`@PreAuthorize("hasRole('ADMIN')")` procura `ROLE_ADMIN`. **Solução:** prefixar `ROLE_` na authority e criar um teste unitário de regressão.

2. Entidade apontando para tabela inexistente

Com a autorização corrigida, a consulta aos indicadores ESG falhou com `ORA-00942`: a entidade

`Collection` usava a tabela `collections`, mas a migration cria `TB_COLLECTIONS`. **Solução:** corrigir o mapeamento e fazer o teste de integração rodar com `ddl-auto=validate`. Agora qualquer divergência entre entidades e migrations quebra o pipeline antes do deploy (o teste falha com *missing table*).

3. Testes dependiam de um banco Oracle

O único teste existente (`contextLoads`) exigia um Oracle acessível, o que inviabilizava o CI. **Solução:** separar testes unitários (Mockito, sem banco, rodam no `mvn test`) de testes de integração (`*IT.java`, via Failsafe) e usar um *service container* Oracle 23 Free no GitHub Actions.

4. Variável de ambiente do segredo JWT ignorada

A propriedade usava `${JWT.SECRET:fiap}`, então a variável `JWT_SECRET` definida no container nunca era lida e todos os ambientes usavam o segredo padrão. **Solução:** corrigir para `${JWT_SECRET:fiap}` e injetar segredos diferentes por ambiente.

5. Oracle na nuvem e custo

Serviços PaaS gratuitos (Render, App Service) não oferecem Oracle e têm pouca memória para rodá-lo em container. **Solução:** uma VM B2ms (8 GB + 4 GB de swap via cloud-init) hospedando os dois ambientes isolados por projeto Compose, coberta pelo crédito do Azure for Students e desligável com `az vm deallocate`.

6. Saber se o deploy realmente funcionou

`docker compose up` retorna sucesso mesmo que a aplicação caia logo depois. **Solução:** Spring Boot Actuator + `HEALTHCHECK` no Dockerfile; o `deploy.sh` espera o status *healthy* (e imprime os logs se falhar), e o pipeline confirma ambiente/versão via `/actuator/info`.

6. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☒
Dockerfile funcional	☒
docker-compose.yml ou arquivos Kubernetes	☒
Pipeline com etapas de build, teste e deploy	☒
README.md com instruções e prints	☒
Documentação técnica com evidências (PDF ou PPT)	☒
Deploy realizado nos ambientes staging e produção	☒

Estrutura do projeto entregue

```
coleta-plus/
├── Dockerfile
├── docker-compose.yml           # ambiente local (app + Oracle)
├── .env.example
├── deploy/
│   ├── docker-compose.deploy.yml # staging e produção (imagem do GHCR)
│   ├── deploy.sh                 # executado na VM pelo pipeline
│   ├── staging.env.example
│   └── production.env.example
├── infra/
│   ├── create-azure-vm.sh        # cria a VM no Azure
│   └── cloud-init.yml            # instala Docker na VM
├── .github/workflows/
│   ├── ci-cd.yml                 # build → testes → imagem → staging → produção
│   └── deploy.yml                # deploy reutilizável por ambiente
├── src/                          # código-fonte + testes
├── docs/                        # esta documentação + prints
└── README.md
```